

Building the Configuration Manager - First Component

Files to Create

1. requirements.txt

```
numpy>=1.23.0
scipy>=1.9.0
pandas>=1.5.0
h5py>=3.7.0
matplotlib>=3.6.0
pyyaml>=6.0
jsonschema>=4.17.0
numba>=0.56.0
pytest>=7.2.0
pytest-cov>=4.0.0
tqdm>=4.64.0
```

2. config_schema.yaml

```
type: object
required: [simulation, grid, physics, obstacles, boundary_conditions, solver, output, analysis]
properties:
  simulation:
    type: object
    required: [name, output_dir]
    properties:
      name:
        type: string
      output_dir:
        type: string

  grid:
    type: object
    required: [Nx, Ny, dx, dt]
    properties:
      Nx:
        type: integer
        minimum: 10
      Ny:
        type: integer
        minimum: 10
      dx:
        type: number
        exclusiveMinimum: 0
      dt:
```

type: number
exclusiveMinimum: 0

physics:
type: object
required: [reynolds_number, characteristic_length, characteristic_velocity]
properties:
reynolds_number:
type: number
exclusiveMinimum: 0
characteristic_length:
type: number
exclusiveMinimum: 0
characteristic_velocity:
type: number
exclusiveMinimum: 0

obstacles:
type: array
items:
type: object
required: [type]
properties:
type:
type: string
enum: [circle, rectangle, naca]

boundary_conditions:
type: object
required: [inlet, outlet, top, bottom]

solver:
type: object
required: [max_iterations]
properties:
tau:
type: [number, 'null']
minimum: 0.5
max_iterations:
type: integer
minimum: 1
convergence_criterion:
type: number
exclusiveMinimum: 0
save_interval:
type: integer
minimum: 1

output:
type: object
properties:
save_fields:
type: boolean
save_forces:
type: boolean

save_vtk:
type: boolean
save_video:
type: boolean
video_fps:
type: integer
minimum: 1

analysis:
type: object
properties:
force_calculation_method:
type: string
enum: [momentum_exchange]
averaging_start_step:
type: integer
minimum: 0
fft_window:
type: string
enum: [hann, hamming, blackman, rectangular]

3. examples/cylinder_re100.yaml

simulation:
name: "cylinder_Re100_vortex_street"
output_dir: "./results/cylinder_re100/"

grid:
Nx: 800
Ny: 400
dx: 1.0
dt: 1.0

physics:
reynolds_number: 100.0
characteristic_length: 40.0
characteristic_velocity: 0.1

obstacles:

- type: "circle"
center: [200, 200]
radius: 20.0

boundary_conditions:
inlet:
type: "zou_he_velocity"
velocity: [0.1, 0.0]
outlet:
type: "convective"
convection_velocity: 0.1
top:
type: "periodic"
bottom:
type: "periodic"

```

solver:
tau: null
max_iterations: 100000
convergence_criterion: 1.0e-6
save_interval: 100

output:
save_fields: true
save_forces: true
save_vtk: true
save_video: true
video_fps: 30

analysis:
force_calculation_method: "momentum_exchange"
averaging_start_step: 50000
fft_window: "hann"

```

4. config_manager.py

```

"""Configuration Manager for LBM CFD Solver.

```

```

This module handles loading, validating, and distributing simulation parameters.
"""

```

```

import yaml
import os
from pathlib import Path
from typing import Dict, Any, Optional
import jsonschema

```

```

class ConfigManager:
    """Load, validate, and provide access to simulation configuration."""

```

```

def __init__(self, config_path: str):
    """Initialize configuration manager.

    Args:
        config_path: Path to YAML configuration file
    """
    self.config_path = config_path
    self.config = self._load_config()
    self._validate_config()
    self._compute_derived_parameters()
    self._create_output_directories()

def _load_config(self) -> Dict[str, Any]:
    """Load YAML configuration file.

    Returns:
        Configuration dictionary

    Raises:

```

```

        FileNotFoundError: If config file doesn't exist
        yaml.YAMLError: If YAML is malformed
    """
    if not os.path.exists(self.config_path):
        raise FileNotFoundError(f"Configuration file not found:
{self.config_path}")

    with open(self.config_path, 'r') as f:
        return yaml.safe_load(f)

def _validate_config(self):
    """Validate configuration against schema.

    Raises:
        ValueError: If configuration is invalid
    """
    # Load schema
    schema_path = Path(__file__).parent / 'config_schema.yaml'

    if not schema_path.exists():
        print(f"Warning: Schema file not found at {schema_path}.
Skipping validation.")
        return

    with open(schema_path, 'r') as f:
        schema = yaml.safe_load(f)

    try:
        jsonschema.validate(instance=self.config, schema=schema)
        print("✓ Configuration validated successfully")
    except jsonschema.exceptions.ValidationError as e:
        raise ValueError(f"Configuration validation failed:
{e.message}")

def _compute_derived_parameters(self):
    """Compute derived parameters (nu, tau, cs2, etc.)."""
    physics = self.config['physics']
    solver = self.config.get('solver', {})

    # Compute kinematic viscosity from Reynolds number
    U = physics['characteristic_velocity']
    L = physics['characteristic_length']
    Re = physics['reynolds_number']
    nu = U * L / Re

    # Speed of sound squared (D2Q9)
    cs2 = 1.0 / 3.0

    # Compute relaxation time from viscosity
    if 'tau' not in solver or solver.get('tau') is None:

```

```

        tau = nu / cs2 + 0.5
        self.config['solver']['tau'] = tau
    else:
        tau = solver['tau']
        nu_computed = cs2 * (tau - 0.5)
        if abs(nu - nu_computed) > 1e-6:
            print(f"Warning: Provided tau={tau} gives
nu={nu_computed:.6f}, "
                  f"but Re={Re} requires nu={nu:.6f}")

    # Store derived parameters
    self.config['derived'] = {
        'nu': nu,
        'tau': self.config['solver']['tau'],
        'cs2': cs2,
        'cs': cs2**0.5
    }

    print(f"✓ Computed derived parameters:")
    print(f"  - Kinematic viscosity (nu): {nu:.6f}")
    print(f"  - Relaxation time (tau):
{self.config['solver']['tau']:.6f}")
    print(f"  - Speed of sound (cs): {cs2**0.5:.6f}")

def _create_output_directories(self):
    """Create output directory structure."""
    output_dir = Path(self.config['simulation']['output_dir'])

    subdirs = ['fields', 'forces', 'vtk', 'videos', 'figures', 'data']
    for subdir in subdirs:
        (output_dir / subdir).mkdir(parents=True, exist_ok=True)

    print(f"✓ Created output directories in {output_dir}")

def get(self, key_path: str, default: Optional[Any] = None) -> Any:
    """Get configuration value by dot-separated path.

    Args:
        key_path: Dot-separated path (e.g., 'grid.Nx')
        default: Default value if key not found

    Returns:
        Configuration value

    Example:
        >>> config.get('grid.Nx')
        800
        >>> config.get('nonexistent.key', 42)
        42
    """

```

```

    keys = key_path.split('.')
    value = self.config

    for key in keys:
        if isinstance(value, dict) and key in value:
            value = value[key]
        else:
            return default

    return value

def save_metadata(self):
    """Save configuration + derived parameters for reproducibility."""
    output_dir = Path(self.config['simulation']['output_dir'])
    metadata_path = output_dir / 'config_used.yaml'

    with open(metadata_path, 'w') as f:
        yaml.dump(self.config, f, default_flow_style=False)

    print(f"✓ Configuration saved to {metadata_path}")

def __repr__(self) -> str:
    """String representation."""
    return (f"ConfigManager(name='{self.get('simulation.name')}', "
            f"grid={self.get('grid.Nx')}\x{self.get('grid.Ny')}, "
            f"Re={self.get('physics.reynolds_number')})")

```

Verification in Jupyter Notebook

Create a notebook with these cells:

Cell 1: Setup and Installation

Install dependencies (if needed)

!pip install pyyaml jsonschema

```
import sys
import os
from pathlib import Path
```

Add parent directory to path if running from subdirectory

```
sys.path.insert(0,  
str(Path.cwd().parent))
```

```
print("✓ Environment ready")
```

Cell 2: Import and Basic Test

```
from config_manager import ConfigManager
```

Test 1: Load valid configuration

```
print("="*60)
print("TEST 1: Loading Valid Configuration")
print("="*60)

try:
    config = ConfigManager('examples/cylinder_re100.yaml')
    print("\n✓ Configuration loaded successfully")
    print(f" {config}")
except Exception as e:
    print(f"✗ Error: {e}")
```

Cell 3: Test Configuration Access

```
print("\n" + "="*60)
print("TEST 2: Configuration Value Access")
print("="*60)
```

Test dot-notation access

```
tests = [  
('grid.Nx', 800),
```

```
( 'grid.Ny', 400),
( 'physics.reynolds_number', 100.0),
( 'physics.characteristic_velocity', 0.1),
( 'physics.characteristic_length', 40.0),
]
```

```
for key_path, expected in tests:
    value = config.get(key_path)
    status = "✓" if value == expected else "✗"
    print(f"{status} {key_path}: {value} (expected {expected})")
```

Test default values

```
default_test = config.get('nonexistent.key', 'DEFAULT')
print(f'{ "✓" if default_test == "DEFAULT" else "✗" } Default value handling: {default_test}')
```

Cell 4: Test Derived Parameters

```
print("\n" + "="*60)
print("TEST 3: Derived Parameter Computation")
print("="*60)
```

Get derived parameters

```
nu = config.get('derived.nu')
tau = config.get('derived.tau')
cs2 = config.get('derived.cs2')
```

Validate $nu = U * L / Re$

```
U = config.get('physics.characteristic_velocity')
L = config.get('physics.characteristic_length')
Re = config.get('physics.reynolds_number')

nu_expected = U * L / Re
nu_error = abs(nu - nu_expected)

print(f"Kinematic viscosity (nu):")
print(f" Computed: {nu:.6f}")
print(f" Expected: {nu_expected:.6f}")
print(f" Error: {nu_error:.2e}")
print(f" Status: { '✓ PASS' if nu_error < 1e-6 else '✗ FAIL' }")
```

Validate $tau = nu / cs2 + 0.5$

```
tau_expected = nu / cs2 + 0.5
tau_error = abs(tau - tau_expected)
```

```

print(f"\nRelaxation time (tau):")
print(f" Computed: {tau:.6f}")
print(f" Expected: {tau_expected:.6f}")
print(f" Error: {tau_error:.2e}")
print(f" Status: {'✓ PASS' if tau_error < 1e-6 else '✗ FAIL'}")

```

Cell 5: Test Output Directory Creation

```

print("\n" + "="*60)
print("TEST 4: Output Directory Structure")
print("="*60)

output_dir = Path(config.get('simulation.output_dir'))
required_subdirs = ['fields', 'forces', 'vtk', 'videos', 'figures', 'data']

all_exist = True
for subdir in required_subdirs:
    path = output_dir / subdir
    exists = path.exists() and path.is_dir()
    status = "✓" if exists else "✗"
    print(f"{status} {subdir}/ directory: {'exists' if exists else 'MISSING'}")
    all_exist = all_exist and exists

print(f"\n{'✓ All directories created' if all_exist else '✗ Some directories missing'}")

```

Cell 6: Test Invalid Configuration

```

print("\n" + "="*60)
print("TEST 5: Invalid Configuration Handling")
print("="*60)

```

Create a temporary invalid config

```

invalid_config_content = """
simulation:
name: "test"
output_dir: "./test_output/"

grid:
Nx: 5 # Too small (minimum is 10)
Ny: 5
dx: 1.0
dt: 1.0

physics:
reynolds_number: -10 # Negative (invalid)
characteristic_length: 40.0
characteristic_velocity: 0.1
"""

```

```

invalid_config_path = 'test_invalid_config.yaml'
with open(invalid_config_path, 'w') as f:
    f.write(invalid_config_content)

try:
    invalid_config = ConfigManager(invalid_config_path)
    print("✗ FAIL: Should have raised validation error")
except ValueError as e:
    print(f"✓ PASS: Correctly rejected invalid configuration")
    print(f"Error message: {str(e)[:100]}...")
finally:
    # Cleanup
    if os.path.exists(invalid_config_path):
        os.remove(invalid_config_path)

```

Cell 7: Test Save Metadata

```

print("\n" + "="*60)
print("TEST 6: Metadata Saving")
print("="*60)

config.save_metadata()

metadata_path = output_dir / 'config_used.yaml'
if metadata_path.exists():
    print("✓ Metadata file created")

# Verify content
with open(metadata_path, 'r') as f:
    saved_config = yaml.safe_load(f)

# Check if derived parameters are saved
if 'derived' in saved_config:
    print("✓ Derived parameters included in metadata")
    print(f"  - nu: {saved_config['derived']['nu']:.6f}")
    print(f"  - tau: {saved_config['derived']['tau']:.6f}")
else:
    print("✗ Derived parameters missing from metadata")

else:
    print("✗ Metadata file not created")

```

Cell 8: Summary

```

print("\n" + "="*60)
print("VERIFICATION SUMMARY")
print("="*60)

print("""
✓ All tests completed!

```

The ConfigManager component provides:

1. YAML configuration loading
2. JSON schema validation
3. Derived parameter computation (nu, tau, cs2)
4. Output directory creation
5. Dot-notation configuration access
6. Metadata saving for reproducibility

Next Steps:

- Build [lattice.py](#) (D2Q9 lattice definition)
- Build `geometry_builder.py` (obstacle generation)
- Build `lbm_core.py` (collision and streaming)

Run all tests with: `pytest tests/test_config_manager.py -v`
"""

Quick Start Commands

1. Create project structure

```
mkdir -p classical_ca_cfd/{examples,tests,scripts}  
cd classical_ca_cfd
```

2. Create files

```
touch requirements.txt  
touch config_schema.yaml  
touch config_manager.py  
mkdir -p examples  
touch examples/cylinder_re100.yaml
```

3. Copy content from above into each file

4. Install dependencies

```
pip install -r requirements.txt
```

5. Run verification notebook

jupyter notebook verify_config_manager.ipynb

Expected Output

When you run the notebook, you should see:

- ✓ All 6 tests passing
- Configuration validated successfully
- Derived parameters computed correctly ($\nu=0.04$, $\tau\approx 0.62$)
- Output directories created
- Invalid configurations properly rejected
- Metadata saved with derived parameters

Success Criteria

- ✓ Configuration loads without errors
- ✓ All parameter access tests pass
- ✓ Derived parameters match analytical values ($\nu = 0.04 \pm 1e-6$)
- ✓ Output directories created
- ✓ Invalid configs properly rejected
- ✓ Metadata saved with derived parameters

Once these tests pass, you're ready to build the next component ([lattice.py](#))!